Projet : Serveur Cloud , Plateforme web sécurisée pour une PME

I. Présentation du projet :

Le projet livre un serveur Ubuntu sur VM VMware qui sert un portail interne en HTTPS derrière Nginx, filtré par UFW, avec sauvegarde quotidienne et contrôle de santé toutes les 15 minutes.

Besoin client : Une PME veut héberger un site vitrine et une petite application interne (admin) en HTTPS, avec un accès admin sécurisé, une supervision minimale et des sauvegardes. La solution doit être fiable, maintenable et peu coûteuse .

Objectifs :

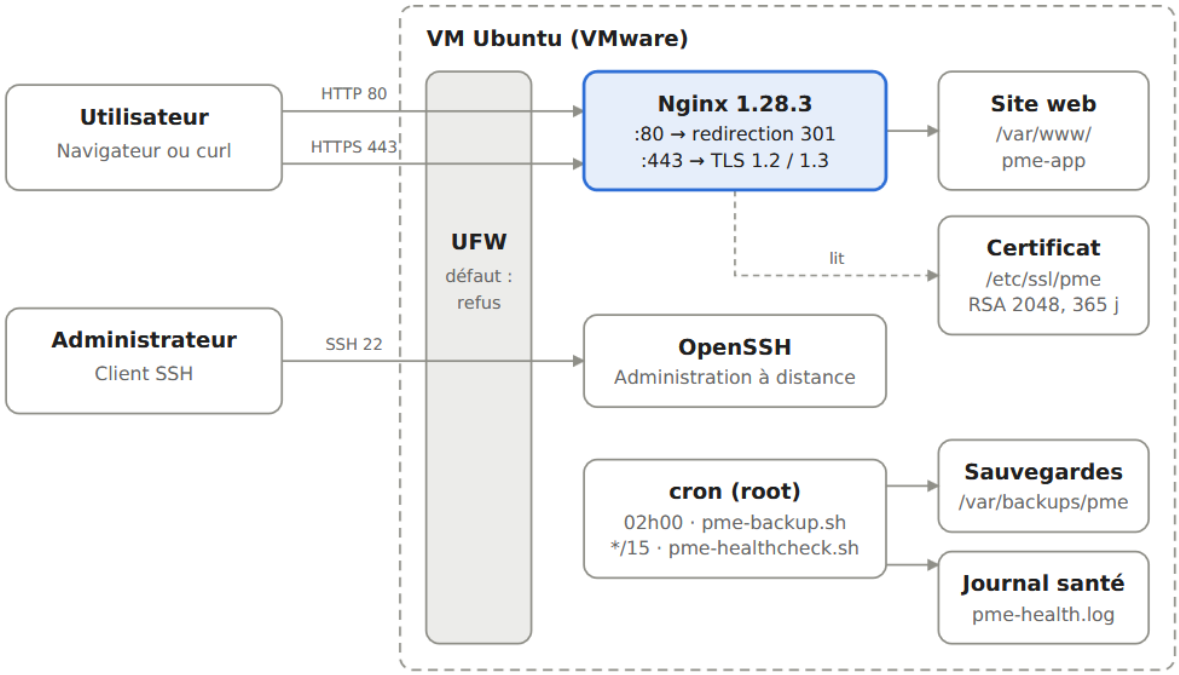
- Déployer une stack « prod-like » sur une VM Linux.
- Mettre en place un reverse proxy et le chiffrement TLS.
- Sécuriser l'administration (SSH, pare-feu, mises à jour).
- Fournir une supervision, des sauvegardes et une procédure de restauration testée.

Périmètre réalisé :

Brique	Réalisation	Emplacement
Système	Ubuntu sur VMware, paquets à jour	VM tahiry-VMware-Virtual-Platform
Pare-feu	UFW : entrant refusé par défaut, ports 22, 80, 443 ouverts	ufw status verbose
Application	Page statique « Portail Interne PME »	/var/www/pme-app/index.html
Certificat	Auto-signé, RSA 2048 bits, 365 jours, CN pme.local	/etc/ssl/pme/
Serveur web	Nginx 1.28.3, redirection 301 HTTP vers HTTPS, TLS 1.2/1.3, en-têtes de sécurité	/etc/nginx/sites-available/pme-app
Sauvegarde	Archive tar.gz quotidienne à 02h00, rétention 7 jours	/usr/local/bin/pme-backup.sh → /var/backups/pme
Supervision	Contrôle HTTP toutes les 15 min, journalisé	/usr/local/bin/pme-healthcheck.sh → /var/log/pme-health.log

II. Architecture et schéma réseau

Tout le trafic entrant est refusé par UFW sauf trois ports : 80 (redirigé vers HTTPS), 443 (portail chiffré) et 22 (administration SSH).



Nginx redirige le port 80 vers 443 puis sert le portail avec le certificat de /etc/ssl/pme . Le healthcheck interroge https://localhost depuis la VM ; le trafic sortant (apt, NTP) est autorisé .

Port	Protocole	Source	Destination	Rôle	Exposition
433	TCP / HTTPS	Tous	Nginx	Portail chiffré (TLS 1.2/1.3)	Public
80	TCP / HTTP	Tous	Nginx	Redirection 301 vers 443 uniquement	Public
44	TCP / SSH	Tous	OpenSSH	Administration	À limiter au réseau admin
Autres	-	Tous	-	Bloqués par la politique deny incoming	Fermé

III. Stack techniques

La stack privilégie des services natifs Ubuntu : peu de composants, aucun coût de licence, et une maintenance à la portée d'une PME sans équipe dédiée .

Docker ou services natifs

Nous avons choisi d'installer les services directement sur le système, avec les paquets apt, plutôt que de passer par Docker. L'application se limite à une seule page statique : Docker aurait ajouté une couche technique supplémentaire sans réel bénéfice. Les mises à jour se font simplement avec apt. La conteneurisation avec Docker reste une évolution envisagée pour la suite du projet.

Reverse proxy

Nginx a été retenu comme serveur web et reverse proxy, de préférence à Caddy. C'est un standard du marché, très bien documenté, léger en mémoire, et sa configuration est explicite et facile à relire. Caddy a l'avantage de gérer le certificat TLS automatiquement, mais il suppose de disposer d'un nom de domaine public, ce qui n'est pas notre cas.

Gestion TLS

Le chiffrement HTTPS repose sur un certificat auto-signé généré avec OpenSSL, plutôt que sur Let's Encrypt (via Certbot ou Caddy). Let's Encrypt exige un nom de domaine public et un port 80 joignable depuis Internet, deux conditions que notre VM ne remplit pas. Le certificat auto-signé chiffre bien les échanges ; sa seule contrepartie est l'avertissement affiché par le navigateur, qui ne reconnaît pas l'autorité qui l'a émis.

Protocoles

Nginx n'accepte que les protocoles TLS 1.2 et TLS 1.3, avec la suite de chiffrement HIGH:!aNULL:!MD5, au lieu des valeurs par défaut. Ce réglage écarte les protocoles anciens et vulnérables (SSLv3, TLS 1.0 et TLS 1.1), ainsi que les chiffrements faibles.

Pare-feu

Le filtrage réseau est assuré par UFW plutôt que par une configuration directe de nftables. UFW est une surcouche simple de nftables : ses règles sont lisibles et rapides à mettre en place, ce qui suffit largement pour les trois règles nécessaires au projet.

Ports exposés

Seuls trois ports sont ouverts : 22, 80 et 443. Le port 443 sert le portail en HTTPS. Le port 80 ne sert qu'à rediriger automatiquement les visiteurs vers HTTPS. Le port 22 permet l'administration à distance par SSH ; il est aujourd'hui ouvert à tous et devra être restreint au réseau d'administration.

Modèle d'accès admin

L'administration se fait par un accès SSH direct, plutôt que via un bastion, un VPN ou une liste blanche d'adresses IP. C'est la solution la plus simple pour une seule VM. Pour renforcer la sécurité, il est recommandé de limiter le port 22 aux adresses IP du réseau d'administration et de protéger la future zone admin web de la même manière.

Supervision

La supervision repose sur un script Bash exécuté par cron toutes les 15 minutes, plutôt que sur un outil comme Netdata, Uptime Kuma ou Grafana. Cette solution ne demande aucune dépendance et laisse une trace exploitable dans un fichier journal. Uptime Kuma serait l'évolution naturelle pour disposer d'une interface graphique et d'alertes.

Sauvegarde

Les sauvegardes sont des archives tar.gz créées chaque nuit sur le serveur, avec une rotation sur 7 jours, plutôt qu'un outil comme Restic ou Borg ou un stockage distant. Le dispositif est simple et facile à vérifier. Sa principale limite est que les archives restent sur la même VM : elles seraient perdues avec elle. Une copie vers un stockage externe est donc à prévoir.

IV.README : installation from scratch

Ces étapes reconstruisent le serveur complet sur une VM Ubuntu vierge, dans l'ordre. Toutes les commandes s'exécutent avec un compte disposant de sudo .

Prérequis : VM Ubuntu (VMware), 1 vCPU, 1 Go de RAM, 10 Go de disque, accès réseau sortant.

Étape 1 : mise à jour et paquets de base

sudo apt update && sudo apt upgrade -y
sudo apt install -y curl htop ufw openssh-server

Si apt update affiche « Release file is not valid yet », l'horloge de la VM est en retard : la resynchroniser avec sudo timedatectl set-ntp true , puis relancer .

Étape 2 : pare-feu UFW

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose

Preuve que ca marche :



Étape 3 : application web

sudo mkdir -p /var/www/pme-app
sudo nano /var/www/pme-app/index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Plateforme PME - Espace Sécurisé</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; background-color: #f4
        .card { background: white; padding: 30px; border-radius: 8px; box-shadow:
        h1 { color: #0056b3; }
        .status { color: green; font-weight: bold; }
    </style>
</head>
<body>
    <div class="card">
        <h1>🏢 Portail Interne PME</h1>
        <p>Statut du service : <span class="status">Opérationnel & Sécurisé (HTTPS
        <hr>
        <p>Bienvenue sur la plateforme web d'entreprise.</p>
    </div>
</body>
</html>
```

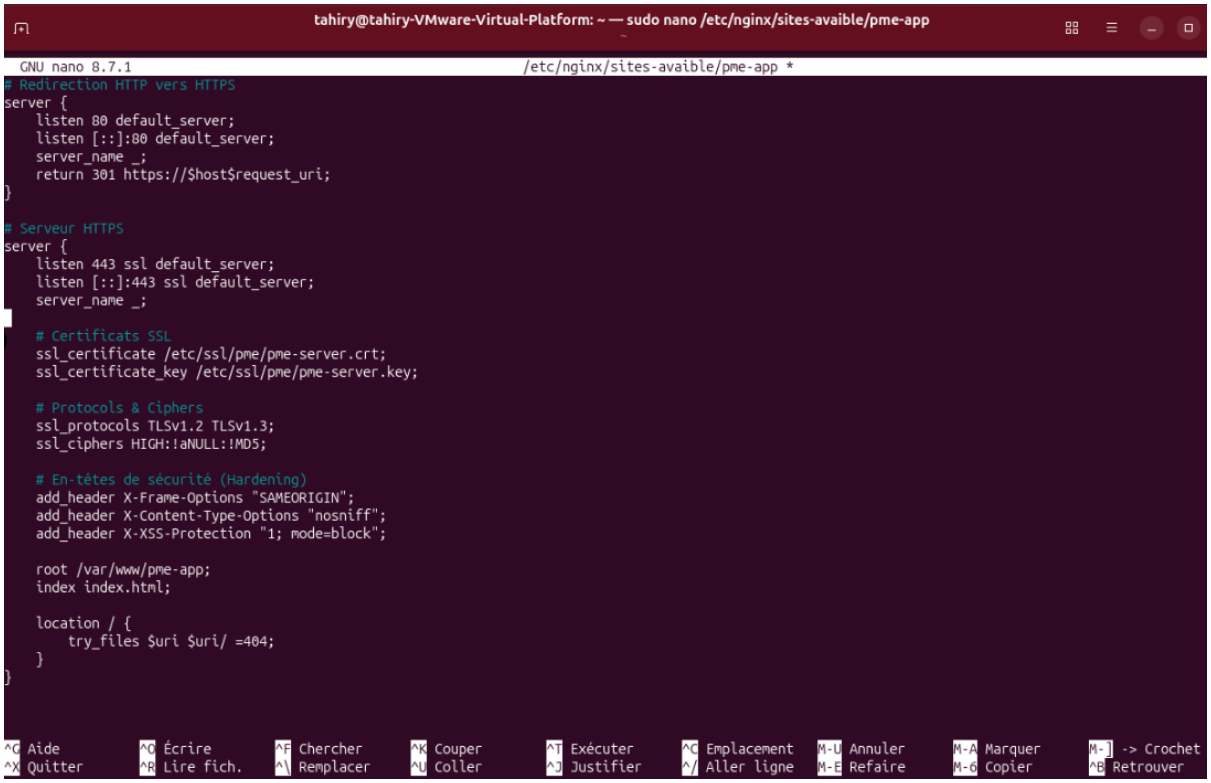
Étape 4 : certificat TLS



Étape 5 : Nginx

Installer Nginx avant d'écrire sa configuration, sinon le dossier sites-available n'existe pas encore.

sudo apt install -y nginx
sudo nano /etc/nginx/sites-available/pme-app



Activer le site, retirer le site par défaut, tester puis recharger :

sudo ln -sf /etc/nginx/sites-available/pme-app /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx

nginx -t doit répondre « syntax is ok » et « test is successful ». Attention à l'orthographe sites-available : une faute de
frappe (sites-avaible) crée un fichier que Nginx ne lit jamais.

Étape 6 : vérification

curl -I http://localhost # attendu : HTTP/1.1 301 Moved Permanently, Location: https://localhost/
curl -k https://localhost # attendu : le code HTML du portail

V.Automatisation : sauvegarde et healthcheck

Deux scripts Bash planifiés par cron assurent le maintien en condition opérationnelle : une sauvegarde chaque nuit à 02h00 et un contrôle HTTP toutes les 15 minutes.

Script de sauvegarde

```
sudo mkdir -p /var/backups/pme
cat << 'EOF' | sudo tee /usr/local/bin/pme-backup.sh
#!/bin/bash
DATE=$(date +%Y-%m-%d_%H%M%S)
BACKUP_DIR="/var/backups/pme"
tar -czf $BACKUP_DIR/pme_app_$DATE.tar.gz /var/www/pme-app /etc/nginx/sites-available/pme-app
find $BACKUP_DIR -type f -mtime +7 -delete
EOF
sudo chmod +x /usr/local/bin/pme-backup.sh
```

Script de healthcheck

```
cat << 'EOF' | sudo tee /usr/local/bin/pme-healthcheck.sh
#!/bin/bash
STATUS=$(curl -k -s -o /dev/null -w "%{http_code}" https://localhost)
if [ $STATUS -eq 200 ] || [ $STATUS -eq 301 ]; then
echo "[$(date)] OK: La plateforme PME répond correctement (HTTP $STATUS)" >> /var/log/pme-health.log
else
echo "[$(date)] ALERTE: Dysfonctionnement détecté (HTTP $STATUS)" >> /var/log/pme-health.log
fi
EOF
sudo chmod +x /usr/local/bin/pme-healthcheck.sh
```

Planification cron

Les deux scripts écrivent dans /var/backups et /var/log , réservés à root : ils doivent donc être planifiés dans la crontab de root (sudo crontab), pas dans celle de l'utilisateur.

```
(sudo crontab -l 2>/dev/null; echo "0 2 * * * /usr/local/bin/pme-backup.sh") | sudo crontab -
(sudo crontab -l 2>/dev/null; echo "*/15 * * * * /usr/local/bin/pme-healthcheck.sh") | sudo crontab - sudo crontab -l
```

Tâche	Fréquence	Expression cron	Sortie
Sauvegarde	Tous les jours à 02h00	0 2 * * *	/var/backups/pme/*.tar.gz
Healthcheck	Toutes les 15 minutes	*/15 * * * *	/var/log/pme-health.log

VI .Runbook d'exploitation

Ce runbook couvre les cinq opérations courantes de l'administrateur : mises à jour, lecture des journaux, sauvegarde, restauration et diagnostic d'incident.

Mises à jour (chaque semaine)

sudo apt update && sudo apt upgrade -y
sudo nginx -t && sudo systemctl reload nginx
[-f /var/run/reboot-required] && echo "Redémarrage nécessaire"

Pour automatiser les correctifs de sécurité : sudo apt install -y unattended-upgrades puis sudo dpkg-reconfigure –plow unattended-upgrades .

Journaux

Journal	Commande	Usage
Santé du portail	tail -n 20 /var/log/pme-health.log	Historique OK / ALERTE
Accès Nginx	sudo tail -f /var/log/nginx/access.log	Requêtes reçues
Erreurs Nginx	sudo tail -n 50 /var/log/nginx/error.log	Causes d'erreurs 4xx/5xx
Service Nginx	sudo journalctl -u nginx --since "1 hour ago"	Démarrages, arrêts, échecs
Pare-feu	sudo tail -f /var/log/ufw.log	Connexions bloquées
Authentification SSH	sudo journalctl -u ssh --since today	Tentatives de connexion

Repérer rapidement les alertes : grep ALERTE /var/log/pme-health.log

Sauvegarde manuelle

sudo /usr/local/bin/pme-backup.sh
ls -lh /var/backups/pme
tar -tzf /var/backups/pme/<archive>.tar.gz # vérifier le contenu sans extraire

Restauration

1. Choisir l'archive à restaurer : ls -lt /var/backups/pme .
2. Vérifier son contenu : tar -tzf /var/backups/pme/.tar.gz .
3. Mettre de côté l'état actuel : sudo mv /var/www/pme-app /var/www/pme-app.old .
4. Restaurer (tar a stocké les chemins sans le / initial, on extrait donc depuis la racine) : sudo tar -xzf /var/backups/pme/.tar.gz -C / .
5. Tester et recharger Nginx : sudo nginx -t && sudo systemctl reload nginx .
6. Valider : curl -k https://localhost doit renvoyer le portail.
7. Une fois validé, supprimer l'ancien état : sudo rm -rf /var/www/pme-app.old

Le certificat (/etc/ssl/pme) n'est pas dans l'archive : en cas de perte, le régénérer (section 4, étape 4) ou l'ajouter au script de sauvegarde.

Diagnostic d'incident : le site ne répond plus

1. Constater : curl -k -I https://localhost (échec ou code ≠ 200) et grep ALERTE /var/log/pme-health.log .
2. État du service : sudo systemctl status nginx .
3. Ports en écoute : sudo ss -tlnp | grep -E ':80|:443' .
4. Configuration : sudo nginx -t (une erreur de syntaxe empêche le démarrage).
5. Cause : sudo tail -n 50 /var/log/nginx/error.log et sudo journalctl -u nginx -n 50 .
6. Pare-feu : sudo ufw status (ports 80 et 443 toujours en ALLOW).
7. Corriger puis redémarrer : sudo systemctl restart nginx , et revérifier avec curl .

Symptôme	Cause probable	Correction
curl: (7) Failed to connect	Nginx arrêté	sudo systemctl start nginx
nginx -t échoue	Erreur de syntaxe, chemin de certificat faux	Corriger le fichier indiqué à la ligne citée
HTTP 404	Mauvais root ou index.html absent	Vérifier /var/www/pme-app
HTTP 502 / 504	Application derrière le proxy arrêtée	Redémarrer l'application amont
OK en local, KO depuis le réseau	Pare-feu ou réseau de la VM	sudo ufw status , mode réseau VMware

VII.Scénario de démonstration et validation

La démonstration suit les cinq points du cahier des charges ; chaque étape a une commande et un résultat attendu vérifiable à l'écran .

	Étape	Commande	Résultat attendu	Validé
1	Accès au site en HTTPS	curl -I http://localhost puis curl -k https://localhost	301 Moved Permanently vers https://localhost/ , puis le HTML du portail	Oui (15/10/2026)
2	Accès à l'admin	ssh <utilisateur>@<ip-vm>	Session ouverte ; zone web admin à ajouter (section 8)	Partiel
3	Afficher la supervision	tail /var/log/pme-health.log et sudo crontab -l	Lignes OK horodatées,	Oui (16/10/2026)

			tâches planifiées listées	
4	Simuler un incident	sudo systemctl stop nginx puis sudo /usr/local/bin/pme-healthcheck.sh	Ligne ALERTE … (HTTP 000) ; diagnostic via le runbook, puis sudo systemctl start nginx	À dérouler
5	Sauvegarde et restauration	sudo /usr/local/bin/pme-backup.sh , suppression de index.html , restauration (section 6)	Archive créée, site cassé (404), puis portail rétabli	À dérouler

Pour l'incident (étape 4), le déroulé conseillé devant le jury : arrêter Nginx, lancer le healthcheck, montrer la ligne ALERTE, exécuter systemctl status nginx pour identifier la cause, redémarrer, relancer le healthcheck et montrer le retour à OK.